DATABASE MANAGEMENT SYSTEMS · WEEK 2 + LAB 2

Modeling the World: Entity-Relationship Design

How to turn a business story into tables that protect themselves from bad data.

Dr.-Ing. Chukwuka Victor Obionwu

TODAY'S JOURNEY

Two hours, six stops

Time	Stop	The question we answer
0:00	Warm-up	What must a marketplace remember?
0:10	Story to entities	What are the "things" in KoboKart?
0:28	Relationships	How do things connect, and how many?
0:48	Diagram to PostgreSQL	How does a drawing become CREATE TABLE?
1:15	Lab 02 (after a break)	Does the database refuse bad data?
1:50	Exit quiz	What stuck?

Every stop has a short exercise. Answers appear on the next click.

LEARNING OBJECTIVES

By the end of today you can…

1 · READ A BUSINESS STORY

Pull out the entities, attributes and relationships hidden in it.

2 · DRAW AND READ ER DIAGRAMS

Use crow's foot notation for 1:1, 1:N and N:M relationships.

3 · MAP DIAGRAM TO TABLES

Apply three rules: entity to table, 1:N to foreign key, N:M to junction table.

4 · LET THE DATABASE ENFORCE RULES

Write PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK constraints in PostgreSQL.

…and in the lab, watch PostgreSQL reject bad data because your schema says so.

HOW TODAY WORKS

Four signs you will see on the slides

YOUR TURN

Guess, draw or calculate before the answer appears. Wrong guesses are welcome.

NEW WORD

A term explained in one plain line the first time it appears. No need to ask.

QUICK CHECK

A multiple-choice or true/false question. Vote with fingers: 1, 2, 3 or 4.

STUCK?

Completely normal. Write your question down; we come back to every one.

WARM-UP · YOUR TURN

What must a marketplace remember?

A friend wants an online market where several sellers can list phones and chargers, and shoppers can buy them. The system has a memory problem: it forgets everything unless we tell it what to store.

YOUR TURN (90 seconds)
On paper, list every fact the system must remember so that a shopper can buy a phone from a seller. Then circle the facts that belong together.

Most lists fall into three groups. Who: shoppers and sellers (name, email). What: products (title, price, stock). Purchases: orders (date, status) and what was in them (quantity). You just did conceptual design. Each group is an **entity**.

STORY TO ENTITIES · STEP 1

Why draw before we build?

Nobody pours concrete before the blueprint. A mistake on paper costs a rubber; a mistake in a finished building costs a wall.

LAYER 1 · CONCEPTUAL DESIGN (ER DIAGRAM)
Boxes and lines in business language: entities and relationships.

LAYER 2 · LOGICAL DESIGN (RELATIONAL SCHEMA)
Tables with primary keys and foreign keys, in database language.

LAYER 3 · THE BUILD (DDL)
CREATE TABLE statements that PostgreSQL runs.

NEW WORDS

Schema: the plan of all tables and how they connect.

DDL (Data Definition Language): SQL commands that create structure, like CREATE TABLE.

ER: Entity-Relationship.

YOUR TURN: which is cheaper to fix, a wrong line on the diagram or a wrong table holding a million rows?

STORY TO ENTITIES · STEP 2

Meet KoboKart

KoboKart is a multi-vendor marketplace. **Users** register as either **customers** or **merchants**. A merchant **lists** many **products**. Each product **belongs to** one merchant and has a **title**, a **stock level** and a **unit price**. A customer **adds** products from several merchants to a **cart**. At checkout, an **order** is generated that **contains** one or more **order items**.

YOUR TURN (2 min): copy this paragraph. Underline the things in one colour and the linking verbs in another. Compare with a neighbour. We have already coloured our answer: things are teal, verbs are gold.

STORY TO ENTITIES · STEP 3

Three kinds of words, three ER pieces

In the story	Becomes	KoboKart example
A noun that has facts about it	Entity	User, Product, Order
A fact about that noun	Attribute	email, price, stock level
A verb linking two nouns	Relationship	a merchant lists products

NEW WORDS

Entity: a kind of thing the system must remember. **Attribute**: one fact about an entity. **Relationship**: how two entities are connected.

YOUR TURN: “price” is a noun too. Why is it an attribute and not an entity?

Price has no facts of its own. It only describes a product. If a noun needs its own list of facts, it is an entity.

QUICK CHECK 1 · VOTE WITH FINGERS

Which of these should be its own entity?

A. A product’s price

B. A product

C. A user’s email

D. A product’s stock level

B. A product. It has its own facts (title, price, stock) and its own identity. A, C and D are single facts, so they are attributes.

STORY TO ENTITIES · STEP 4

An entity, its attributes and its key

USERS

- **user_id** (key)
- email
- full_name
- user_type

NEW WORDS

Entity = the rectangle. **Attribute** = a line inside it.

Primary key (PK): the attribute that identifies exactly one row, like a student number.

YOUR TURN: list four attributes for the **Product** entity, and mark its key.

product_id (key), title, price, stock_quantity. The link to the merchant comes later, from the relationship, not from the list of attributes.

QUICK CHECK 2 · VOTE WITH FINGERS

Which attribute is the best primary key for Users?

A. full_name

B. email

C. user_id (a generated UUID)

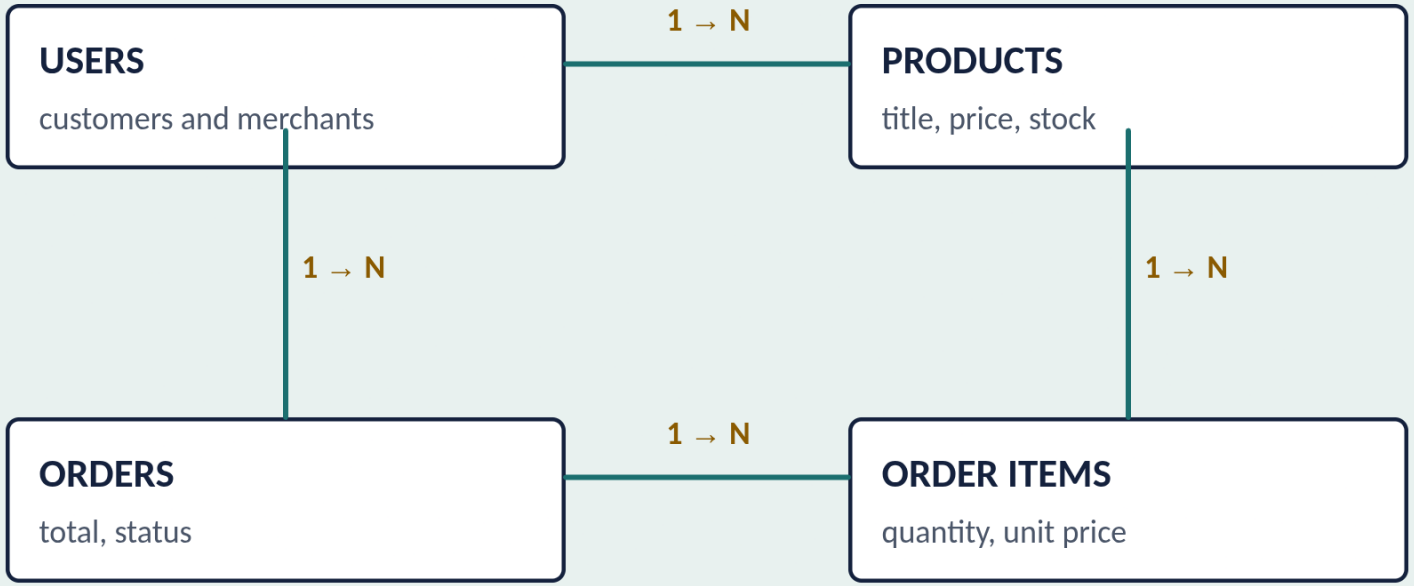
D. user_type

C. user_id. A good key is unique, never empty and never changes.

Two people can share a name (A). Many users share a type (D). Email is unique (B) but people change addresses, so we keep email as a separate UNIQUE column instead.

STORY TO ENTITIES · THE WHOLE PICTURE

KoboKart on one page



READ IT OUT LOUD

A merchant (user) **lists** many products.

A customer (user) **places** many orders.

An order **contains** many order items.

A product **appears in** many order items.

Orders and Products are really many-to-many. Order Items is the bridge between them, which we explain in Part 3.

RELATIONSHIPS · STEP 1

How many of these connect to how many of those?

1 : 1

one to one

User and their settings

1 : N

one to many

Merchant and products

N : M

many to many

Orders and products

NEW WORD

Cardinality: how many of one entity can connect to one of the other. It is the “1”, the “N” and the “M” in the labels above.

YOUR TURN: match each to 1:1, 1:N or N:M: (a) a person and their passport, (b) a mother and her children, (c) students and courses.

(a) 1:1 (b) 1:N (c) N:M. A student takes many courses and a course has many students.

RELATIONSHIPS · STEP 2 · ONE TO MANY

One merchant lists many products

Merchant	Name
M1	Amara
M2	Tunde

Product	Title	Sold by	Price
P1	Phone	M1	250,000
P2	Charger	M1	15,000
P3	Tablet	M2	180,000

READ IT IN BOTH DIRECTIONS

Left to right: one merchant (M1) has **many** products (P1, P2). Right to left: each product has **exactly one** merchant.

YOUR TURN: which table carries the “Sold by” pointer, the merchant side or the product side?

The product side, the “many” side. A product has one merchant, so it fits in one cell. Hold on to this: it becomes mapping Rule 2.

RELATIONSHIPS · STEP 3

Crow’s foot: four symbols to learn



Exactly one, no more, no less



Zero or one, optional, at most one



One or many, at least one



Zero or many, optional, any number

NEW WORDS

Crow’s foot: the three-pronged end that means “many”.

Bar = one. **Circle** = zero (optional).

The thick edge on the right is the entity box. The symbol at that end says how many of that entity connect.

WORKED EXAMPLE · WE READ A LINE TOGETHER

How to read a relationship line



LEFT TO RIGHT
One merchant has **zero or many** products. (A new merchant may have none yet.)

RIGHT TO LEFT
A product belongs to **exactly one** merchant.

YOUR TURN: an Order is linked to Order Items with “exactly one” at the Order end and “one or many” at the Order Items end. Say it in both directions.

An order has one or many items; each item belongs to exactly one order. No empty orders.

QUICK CHECK 3 · VOTE WITH FINGERS

Name the cardinality

"A customer can place many orders. Every order is placed by exactly one customer." What is the cardinality between Customer and Order?

A. 1:1

B. 1:N

C. N:M

D. N:1

B. 1:N (one customer, many orders). D is the same relationship read from the other end, but we always state it with the "one" side first.

RELATIONSHIPS · STEP 4 · ONE TO ONE

One user has exactly one settings record

User	Email
U1	amara@kobokart.test
U2	ife@kobokart.test

Settings	User	Language	Alerts
S1	U1	English	on
S2	U2	Yoruba	off

Each user appears **once** in the settings table, and each settings row points to **one** user. Why separate it? It keeps rarely used or private details away from the main table.

PREVIEW OF THE MAPPING

Same trick as 1:N: put a pointer (foreign key) in one table, then add **UNIQUE** to it so nobody can point twice. UNIQUE is what turns “many” into “one”.

YOUR TURN: think of one more 1:1 example from daily life.

RELATIONSHIPS · STEP 5 · MANY TO MANY

An order holds many products, and a product is in many orders

A tempting shortcut: store the list of products inside the order row.

Order	Products (one cell)	Total
O1	Phone, Charger, Charger	280,000
O2	Phone	250,000

YOUR TURN (1 min): find three problems with this design. Think: counting, prices, deleting a product.

1. You cannot count or search by product without splitting text apart.
2. There is no place for the quantity or the price paid for each product.
3. The database cannot check that every named product really exists or still has stock.

RELATIONSHIPS · STEP 6 · THE FIX

Break N:M into two 1:N with a junction table



order_id	product_id	quantity	unit_price
O1	P1 (Phone)	1	250,000
O1	P2 (Charger)	2	15,000
O2	P1 (Phone)	3	250,000

NEW WORDS

Junction table: a table whose job is to connect two others. Each row is one pairing.

YOUR TURN: how many rows does the junction table need for an order with 3 different products?

3 rows, one per product on the order. Quantity and price paid live here, because they describe the pairing.

QUICK CHECK 4 · TRUE OR FALSE? THUMBS UP OR DOWN

Four statements about relationships

#	Statement	T or F?
1	An N:M relationship can be stored with a single foreign key.	
2	In a 1:N relationship, the pointer goes on the “N” side.	
3	Quantity belongs in the junction table.	
4	A merchant with no products yet breaks the diagram.	

1 · False: use a junction

2 · True: many side holds it

3 · True: it describes the pairing

4 · False: “zero or many”

PART 3 · FROM DIAGRAM TO POSTGRESQL

Three rules turn boxes into tables

RULE 1
Entity becomes a table

RULE 2
1:N becomes a foreign key

RULE 3
N:M becomes a junction table

YOUR TURN: which rule handles Merchant and Products?

Rule 2. One merchant, many products: the pointer goes on the products table.

MAPPING · RULE 1 · ENTITY TO TABLE

Every entity gets a table and a primary key

```
CREATE TABLE users (
  user_id UUID PRIMARY KEY
    DEFAULT gen_random_uuid(),
  email VARCHAR(255) NOT NULL UNIQUE,
  full_name VARCHAR(100) NOT NULL,
  user_type VARCHAR(20) NOT NULL
    CHECK (user_type IN
      ('CUSTOMER','MERCHANT','ADMIN'))
);
```

Piece	In plain words
PRIMARY KEY	Identifies one row; unique, never empty
DEFAULT gen_random_uuid()	The database invents the id
NOT NULL	Must be filled in
UNIQUE	No two users share an email
CHECK (… IN …)	Only three allowed user types

UUID = a long random unique id. VARCHAR(255) = text up to 255 characters. Each attribute from the diagram became a column.

MAPPING · THE GUARD RAILS

Six constraints and the bad data each one blocks

Constraint	Rule in plain words	Bad data it stops
PRIMARY KEY	Row is identifiable	Two rows with the same id, or no id
FOREIGN KEY	Value must exist elsewhere	A product for a merchant that does not exist
NOT NULL	Must be filled in	A product with no title
UNIQUE	No duplicates allowed	Two accounts with one email
CHECK	Custom yes/no test	A negative price
DEFAULT	Fill in if missing	An empty stock count (becomes 0)

NEW WORD

Constraint: a rule the database itself enforces, so no program, however buggy, can break it.

QUICK CHECK 5 · FILL THE BLANKS ON PAPER

Design a new table: categories

Each category has an id (auto-generated, identifies the row) and a name (up to 50 characters, required, no two categories with the same name).

```
CREATE TABLE categories (
  category_id UUID __(1)__ DEFAULT gen_random_uuid(),
  name VARCHAR(50) __(2)__ __(3)__
);
```

Write your three answers, then click.

ANSWERS

(1) PRIMARY KEY

(2) NOT NULL

(3) UNIQUE

(2) and (3) can swap places.

MAPPING · RULE 2 · ONE TO MANY

Copy the “one” side’s key into the “many” side

```
CREATE TABLE products (
  product_id UUID PRIMARY KEY
    DEFAULT gen_random_uuid(),
  merchant_id UUID NOT NULL
    REFERENCES users(user_id)
    ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  price DECIMAL(10, 2) NOT NULL
    CHECK (price ⩾ 0.00),
  stock_quantity INT NOT NULL DEFAULT 0
    CHECK (stock_quantity ⩾ 0)
);
```

NEW WORDS

Foreign key (FK): a column whose values must match a primary key in another table.

REFERENCES: the SQL word that creates the link.

YOUR TURN: why not put product_id inside the users table instead?

A merchant has many products, and one cell cannot hold many ids. Each product holds its one merchant.

MAPPING · WHAT DOES ON DELETE CASCADE DO? · PREDICT FIRST

Merchant M1 closes their account

Product	Title	merchant_id
P1	Phone	M1
P2	Charger	M1
P3	Tablet	M2

NEW WORD

CASCADE: when the parent row is deleted, delete the child rows that point to it, automatically.

YOUR TURN: run DELETE FROM users WHERE user_id = 'M1'. Which products are left in the table? Write down P1, P2, P3, or a combination.

Only P3 (Tablet). The database removed M1's two products for us, so no product is left pointing at a merchant who does not exist.

QUICK CHECK 6 · VOTE WITH FINGERS

Deleting a product that was already ordered

order_items.product_id is declared REFERENCES products(product_id) with **no** ON DELETE CASCADE. Product P1 appears in an order item. What happens on DELETE FROM products WHERE product_id = 'P1'?

A. Product and its order item are both deleted

B. Product deleted, order item keeps a dangling id

C. PostgreSQL refuses with a foreign key error

D. The order item's product_id becomes NULL

C. Without CASCADE the default is to refuse. PostgreSQL reports: *update or delete on table "products" violates foreign key constraint "order_items_product_id_fkey"*.

That is deliberate: order history must not lose the product it recorded. The same refusal stops deleting a merchant whose products were ever ordered.

MAPPING · RULE 3 · MANY TO MANY

A junction table with two keys that work as one

```
CREATE TABLE order_items (
  order_id UUID NOT NULL
    REFERENCES orders(order_id)
    ON DELETE CASCADE,
  product_id UUID NOT NULL
    REFERENCES products(product_id),
  quantity INT NOT NULL
    CHECK (quantity > 0),
  unit_price DECIMAL(10, 2) NOT NULL
    CHECK (unit_price ⩾ 0.00),
  PRIMARY KEY (order_id, product_id)
);
```

NEW WORD

Composite key: a primary key made of two or more columns. The pair must be unique, though each column alone may repeat.

Two FKs, one to each parent.
CASCADE on orders: deleting an order deletes its lines.
No cascade on products: order history is protected.
CHECKs: no zero or negative quantity.

Create order: users, then products and orders, then order_items. A table can only point at tables that already exist.

QUICK CHECK 7 · PREDICT WHICH INSERT FAILS

Four attempts to add a line to order_items

#	order_id	product_id	quantity	Works?
1	O1	P1	1	
2	O1	P2	2	
3	O2	P1	3	
4	O1	P1	5	

YOUR TURN: attempts run in order. Which ones succeed? Why?

1, 2, 3 succeed. 4 fails: the pair (O1, P1) already exists. PostgreSQL says *duplicate key value violates unique constraint "order_items_pkey"*.

P1 repeats across orders (fine) and O1 repeats across products (fine). Only the pair must be unique. To buy more of the same product, update the quantity of the existing line.

MAPPING · A DESIGN QUESTION

Why store unit_price again in order_items?

Products already has a price column. Yet order_items repeats it. Imagine Amara raises the Phone from 250,000 to 300,000 next month.

WITHOUT unit_price IN THE LINE
Last month's order of 2 phones now shows 2 × 300,000 = 600,000. History rewrote itself.

WITH unit_price IN THE LINE
The line keeps 250,000, so the order still shows 2 × 250,000 = 500,000.

YOUR TURN: what word describes copying a value at the moment of purchase?

A snapshot. It is deliberate copying that protects history, not careless duplication.

RECAP BEFORE THE LAB

From diagram to SQL on one page

In the ER diagram	Rule	In PostgreSQL
Entity	Rule 1	CREATE TABLE with a PRIMARY KEY
1:N	Rule 2	FK column on the N side: REFERENCES parent(key)
N:M	Rule 3	Junction table, two FKs, composite PRIMARY KEY
Business rule	Constraints	NOT NULL, UNIQUE, CHECK, DEFAULT, ON DELETE

YOUR TURN (30 s): cover the right column with your hand. Can you say what each row becomes in SQL?

1:10 – 1:15

5-minute break

Stretch, then get your database ready for the lab.

1 · Open your SQL editor (psql or pgAdmin)

2 · Connect to your practice database

3 · Run: SELECT version();

LAB 02 · KOBOKART SCHEMA MODELING · 1:15 – 1:50

From requirements to a database that says no

WARM-UP · 25 PTS

ER diagram analysis and the users table

Today: 10 minutes

CORE · 45 PTS

DDL with foreign keys, cascades and CHECKs

Today: 15 minutes

BOSS LEVEL · 30 PTS

Multi-vendor order line constraints

Today: start; finish at home

The full lab guide allows 20 + 35 + 40 minutes. In class we do the guided core of each tier; **finish the Boss Level as homework**. Work in pairs: one types, one checks against the lecture, and swap at each tier.

LAB · WARM-UP · 10 MINUTES · 25 POINTS

Task 1 · ER diagram analysis

A. Fill in the cardinality for each pair. Use 1:N, 1:1 or N:M.

Pair	Your answer
Merchant (user) and Products	
Customer (user) and Orders	
Orders and Order Items	
Products and Order Items	
Orders and Products, overall	

B. Create and run the users table:

```
CREATE TABLE users (
  user_id UUID PRIMARY KEY
    DEFAULT gen_random_uuid(),
  email VARCHAR(255) NOT NULL UNIQUE,
  full_name VARCHAR(100) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE
    DEFAULT CURRENT_TIMESTAMP
);
```

The lab's version of users has no user_type column. Add it from the lecture if you want to tell merchants from customers.

LAB · WARM-UP · CHECK YOUR WORK

Answers, then your first test

Pair	Answer
Merchant and Products	1:N
Customer and Orders	1:N
Orders and Order Items	1:N
Products and Order Items	1:N
Orders and Products	N:M, resolved by order_items

TEST THE UNIQUE CONSTRAINT

```
INSERT INTO users (email, full_name)
VALUES ('amara@kobokart.test',
        'Amara Obi');
```

Run it twice. The first succeeds.

YOUR TURN: what will PostgreSQL say the second time? Write your guess before you run it.

ERROR: duplicate key value violates unique constraint “users_email_key”. The database, not your code, refused the duplicate email.

LAB · CORE · 15 MINUTES · 45 POINTS

Task 2 · DDL schema enforcement

```
CREATE TABLE products (
  product_id UUID PRIMARY KEY
    DEFAULT gen_random_uuid(),
  merchant_id UUID NOT NULL
    REFERENCES users(user_id) ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  price DECIMAL(10, 2) NOT NULL
    CHECK (price ⩾ 0.00),
  stock_quantity INT NOT NULL DEFAULT 0
    CHECK (stock_quantity ⩾ 0),
  created_at TIMESTAMP WITH TIME ZONE
    DEFAULT CURRENT_TIMESTAMP
);
```

STEPS

1 · Run the products DDL

2 · Insert one user (the merchant)

3 · Insert two products for them

4 · Run the five tests on the next slide

Why create users first? products points at users, so users must already exist.

LAB · CORE · TRY TO BREAK IT

Predict, run, compare

#	Test	What the database should say
a	Insert a product with price = -5	check constraint “products_price_check”
b	Insert a product with stock_quantity = -1	check constraint “products_stock_quantity_check”
c	Insert a product with a merchant_id that does not exist	foreign key constraint “products_merchant_id_fkey”
d	Insert a product with no title	null value in column “title” violates not-null
e	Delete the merchant who owns the products	DELETE succeeds; their products are gone too

YOUR TURN: cover the right column. Say the outcome for each test, then run them one at a time and compare.

Messages checked on PostgreSQL 16. Test e works as shown only while no order line references those products.

LAB · BOSS LEVEL · START NOW, FINISH AT HOME · 30 POINTS

Task 3 · Multi-vendor order lines

```
-- Step 1: the parent (our design)
CREATE TABLE orders (
  order_id UUID PRIMARY KEY
    DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL
    REFERENCES users(user_id),
  total DECIMAL(10, 2) NOT NULL
    DEFAULT 0 CHECK (total ⩾ 0),
  status VARCHAR(20) NOT NULL
    DEFAULT 'PENDING' CHECK (status IN
    ('PENDING','PAID','SHIPPED','CANCELLED'))
);
```

```
-- Step 2: the lab's order_items
CREATE TABLE order_items (
  order_id UUID NOT NULL
    REFERENCES raw_orders(order_id)
    ON DELETE CASCADE,
  product_id UUID NOT NULL
    REFERENCES products(product_id),
  quantity INT NOT NULL CHECK (quantity > 0),
  unit_price DECIMAL(10, 2) NOT NULL
    CHECK (unit_price ⩾ 0.00),
  PRIMARY KEY (order_id, product_id)
);
```

WATCH THE NAME: the lab's snippet points at **raw_orders**. The table you create must have exactly the name the REFERENCES clause uses, so either name the table raw_orders or change the clause to orders.

LAB · BOSS LEVEL · THREE QUESTIONS TO TEST YOUR DESIGN

Does the schema really do what we want?

Q1 · Multi-vendor. Can one order contain products from two different merchants?

Q2 · Same product twice. Can the same product appear on one order as two separate lines? What should you do instead?

Q3 · Spot the gap. products.merchant_id only has to match some user_id. Can a CUSTOMER be a product's merchant?

Q1 · Yes. A line links a product to an order; the product knows its own merchant.

Q2 · No. The primary key (order_id, product_id) forbids a second line. Increase the quantity instead.

Q3 · Yes, with the lab's DDL. The foreign key only proves the user exists. To close the gap, give merchants their own table, or add a composite foreign key that includes the user type. We confirmed it by inserting such a row.

STUCK? TOTALLY NORMAL

The five most common lab errors

What you see	Likely cause	Fix
relation “users” does not exist	Tables created in the wrong order	Create parents first: users, then products and orders, then order_items
relation “raw_orders” does not exist	Name in REFERENCES differs from your table	Make the two names match
function gen_random_uuid() does not exist	PostgreSQL older than 13	CREATE EXTENSION pgcrypto;
relation “users” already exists	Script run twice	DROP TABLE users CASCADE; then rerun (deletes its data)
violates foreign key constraint	Parent row missing	Insert the user or product first

Still stuck after two minutes? Raise a hand, or write the exact error message on the board’s parking lot.

EXIT QUIZ · 5 MINUTES, ON YOUR OWN

Six questions, one letter each

1. A primary key must be…

A unique and never empty · B unique but may be empty · C allowed to repeat · D always a name

2. In a 1:N relationship the foreign key goes…

A on the 1 side · B on the N side · C on both · D in neither table

3. An N:M relationship is stored with…

A a list in one column · B two keys in one entity · C a junction table · D a CHECK

4. CHECK (price >= 0.00) will…

A sort the prices · B reject a row with a negative price · C make prices unique · D delete cheap products

5. A merchant with no ordered products is deleted; products.merchant_id has ON DELETE CASCADE. Their products…

A block the delete · B are deleted too · C get a NULL merchant · D are untouched

6. PRIMARY KEY (order_id, product_id) means…

A one line per product per order · B a product in only one order · C quantity is always 1 · D order_id alone is unique

EXIT QUIZ · MARK YOUR OWN

Answers and the reason behind each

#	Answer	Why
1	A	It identifies exactly one row, so it must be unique and always filled in.
2	B	The N side holds one pointer per row; the 1 side could not hold many.
3	C	Two 1:N relationships through a table whose rows are the pairings.
4	B	A CHECK is a yes/no test every row must pass; negatives fail it.
5	B	CASCADE deletes the child rows. (If an ordered product existed, the delete would be blocked instead.)
6	A	The pair is unique, so a product can appear once per order, in many orders.

5–6 correct: ready for the next lecture. 3–4: revisit the slides for the ones you missed. 0–2: come to office hours and we will go through it together.

REFLECTION · TAKE THESE HOME

Can you answer these in your own words?

1. Why do we draw an ER diagram before writing any CREATE TABLE?

2. How do you decide which table gets the foreign key in a one-to-many relationship?

3. Why does order_items store unit_price when products already has a price?

MUDDIEST POINT: on a slip of paper, write the one idea from today that is still unclear. We start next time with the most common one. **Homework:** finish the Boss Level and answer Q3 from the Boss slide.

ENTITY-RELATIONSHIP DESIGN

Thank you. Questions?

Today a business story became a diagram, and the diagram became a database that refuses bad data on its own.

Dr.-Ing. Chukwuka Victor Obionwu